Convex constrained-optimization

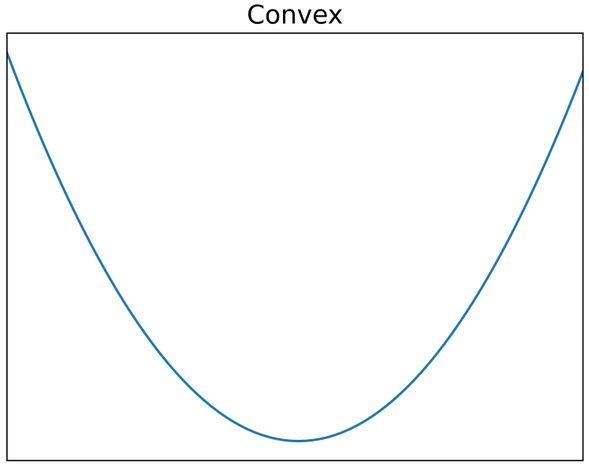
INTRODUCTION TO OPTIMIZATION IN PYTHON



Jasmin Ludolf
Content Developer

datacamp

Convex constrained-optimization



- Constraint: limitation on variables
- Indifference curve: represents a combination of variables
- Find the point on the indifference curve that minimizes or maximizes the objective function

The indifference curve



- Freelance software engineer
- Values work (w) and leisure (l)

Utility function:

- $U(w, l) = w^{0.4} l^{0.6}$

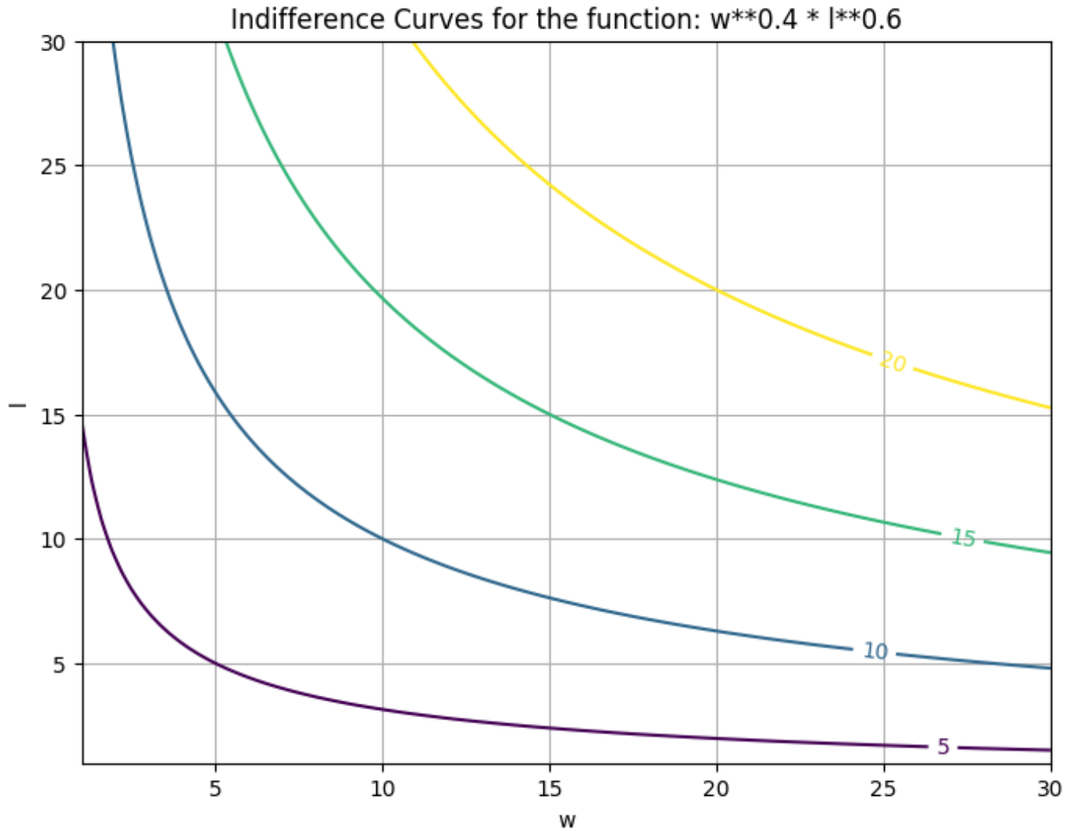
Plotting the indifference curve

```
import numpy as np
import matplotlib.pyplot as plt

w = np.linspace(1, 30, 100)
l = np.linspace(1, 30, 100)
W, L = np.meshgrid(w, l)
F = W**0.4 * L**0.6

plt.figure(figsize=(8, 6))
contours = plt.contour(W, L, F, levels=[5, 10, 15, 20])
plt.clabel(contours)
plt.title('Indifference Curves for the function: w**0.4 * l**0.6')
plt.xlabel('w')
plt.ylabel('l')
plt.grid(True)
plt.show()
```

The indifference curve



- Alexia is indifferent about the specific combination of w and l

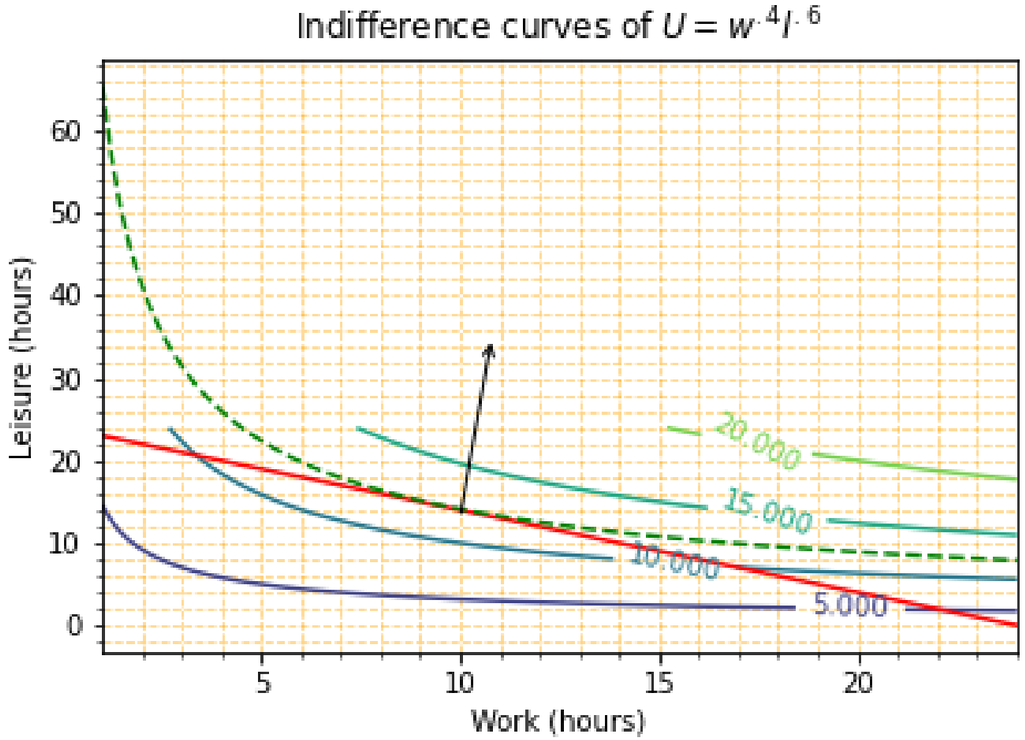
Time constraint

- 24 hours per day
- Linear equality constraint

$$\max_{w,l} w^{0.4} l^{0.6}$$

$$s.t. \quad w + l = 24$$

- Add constraint to the graph:
  - `l = 24 - w`
  - `plt.plot(w, l, color='red')`



Solving with SciPy

```
def utility_function(vars):
    w, l = vars
    return -(w**0.4 * l**0.6)

def constraint(vars):
    return 24 - np.sum(vars)

initial_guess = [12, 12]
constraint_definition = {'type': 'eq', 'fun': constraint}
result = minimize(utility_function, initial_guess, constraints=constraint_definition)
print(result.x)
```

```
[ 9.60001122 14.39998878]
```

Let's practice!

INTRODUCTION TO OPTIMIZATION IN PYTHON

Convex-constrained optimization with inequality constraints

INTRODUCTION TO OPTIMIZATION IN PYTHON



Jasmin Ludolf
Content Developer

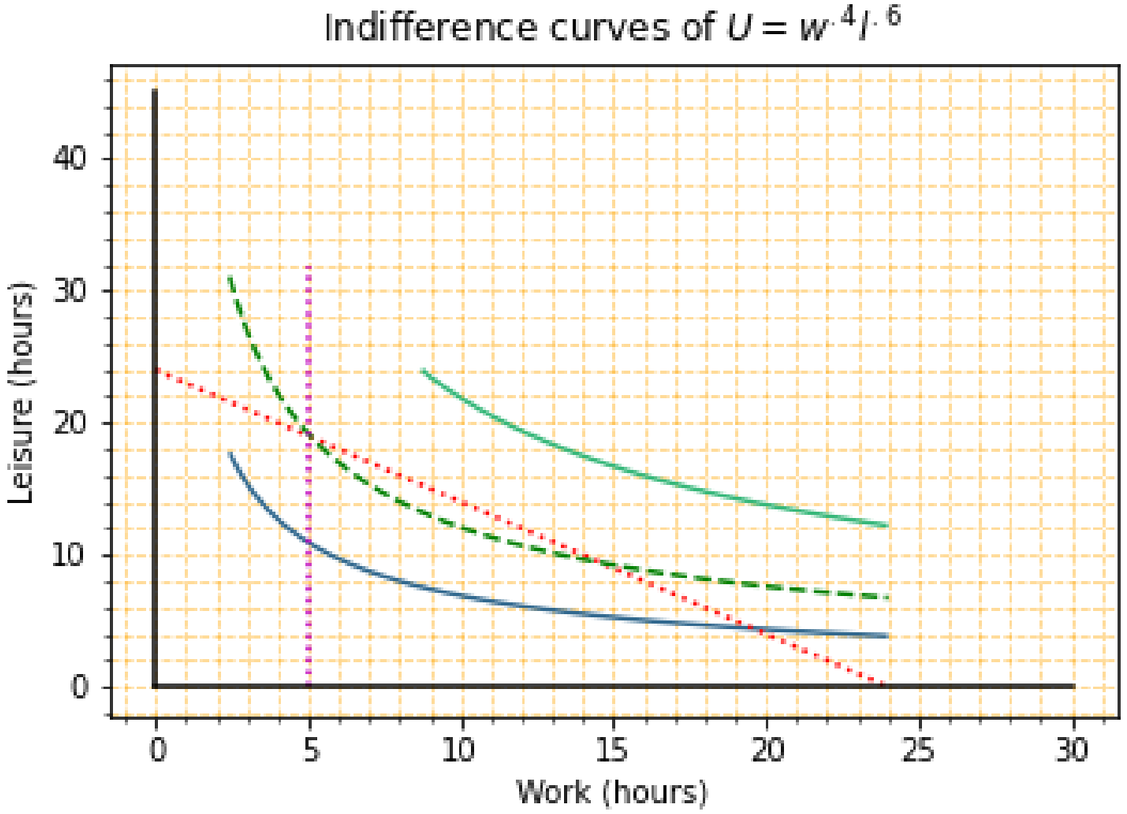
datacamp

Corner versus interior solution

- Alexia only has 5 hours worth of work

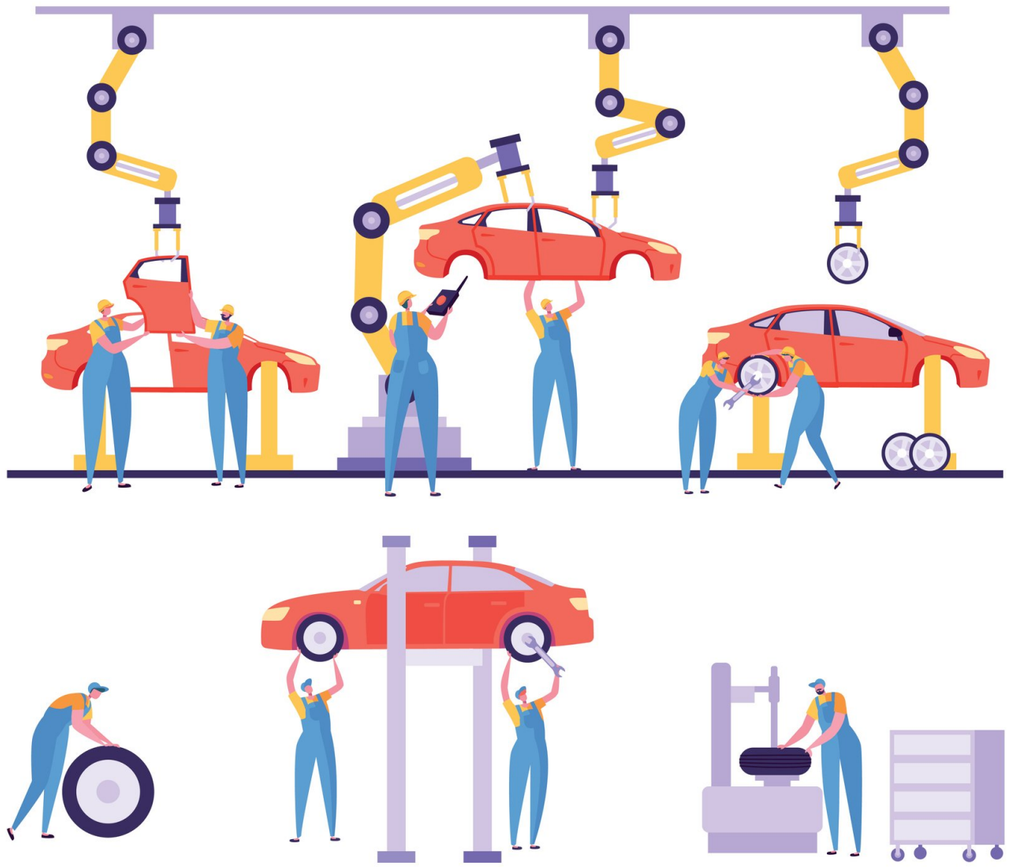
$$w \leq 5$$

- Two constraints:
  - 24 hours in a day
  - 5 hours of work
- The constraints form a corner
- Interior solution is not an intersection



Manufacturer with capacity constraints

- Automaker produces identical cars in two plants A, B
  - Quantities: q_A, q_B
  - Capacity: $q_A \leq 90, q_B \leq 90$
  - Cost: $C_A(q) = 3q, C_B(q) = 3.5q$
- Demand:
  - $P = 120 - Q$
- Contract:
  - $Q \geq 92$
- Objective to maximize profit:
  - $\max \Pi(q_A, q_B)$



Maximizing profit

- Objective:
  - $\Pi = R - C$
    - $R = PQ$

Maximizing profit

- Objective:
  - $\Pi = R - C$
    - $R = PQ = (120 - Q)Q$

Maximizing profit

- Objective:
  - $\Pi = R - C$
    - $R = PQ = (120 - Q)Q = [120 - (q_A + q_B)]\,(q_A + q_B)$
    - $C = C_A + C_B = 3q_A + 3.5q_B$
- Bounds
  - $0 \leq q_A, q_B \leq 90$
- Constraints
  - $Q \geq 92 \Leftrightarrow 92 \leq q_A + q_B$

Formulation

$$\max_{q_A, q_B} R(q_A, q_B) - C(q_A, q_B)$$

$$s.t.$$

$$R(q_A, q_B) = [120 - (q_A + q_B)]\,(q_A + q_B)$$

$$C(q_A, q_B) = 3q_A + 3.5q_B$$

$$0 \leq q_A, q_B \leq 90$$

$$92 \leq q_A + q_B$$

```
from scipy.optimize import minimize,\
        Bounds, LinearConstraint

def R(q):
    return (120 - (q[0] + q[1]
       )) * (q[0] + q[1])
def C(q): return 3*q[0] + 3.5*q[1]
def profit(q): return R(q) - C(q)

bounds = Bounds([0, 0], [90, 90])

constraints = LinearConstraint([1, 1],
                               lb=92)
```

Maximize profit with SciPy

```
result = minimize(lambda q: -profit(q),
                  [50, 50],
                  bounds=bounds,
                  constraints=constraints)
print(result.message)
print(f'The optimal number of cars produced in plant A is: {result.x[0]:.2f}')
print(f'The optimal number of cars produced in plant B is: {result.x[1]:.2f}')
print(f'The firm made: ${-result.fun:.2f}')
```

```
Optimization terminated successfully
The optimal number of cars produced in plant A is: 90.00
The optimal number of cars produced in plant B is: 2.00
The firm made: $2299.00
```

Non-linear contraints in SciPy

```
from scipy.optimize import NonlinearConstraint
import numpy as np

constraints = NonlinearConstraint(lambda q: q[0] + q[1], lb=92, ub=np.inf)

result = minimize(lambda q: -profit(q),
                  [50, 50],
                  bounds=Bounds([0, 0], [90, 90]),
                  constraints=constraints)
```

Solution with NonlinearConstraint

```
print(result.message)
print(f'The optimal number of cars produced in plant A is: {result.x[0]:.2f}')
print(f'The optimal number of cars produced in plant B is: {result.x[1]:.2f}')
print(f'The firm made: ${-result.fun:.2f}')
```

```
Optimization terminated successfully
The optimal number of cars produced in plant A is: 90.00
The optimal number of cars produced in plant B is: 2.00
The firm made: $2299.00
```

- Use `LinearConstraint()` for straightforward, linear problems
- Otherwise, use `NonlinearConstraint()`

Let's practice!

INTRODUCTION TO OPTIMIZATION IN PYTHON

Mixed integer linear programming (MILP)

INTRODUCTION TO OPTIMIZATION IN PYTHON



Jasmin Ludolf
Content Developer

datacamp

Mixed integer linear programming

- MILP
- Optimization technique used when constraint variables are discrete variables

Gowns or tuxedos

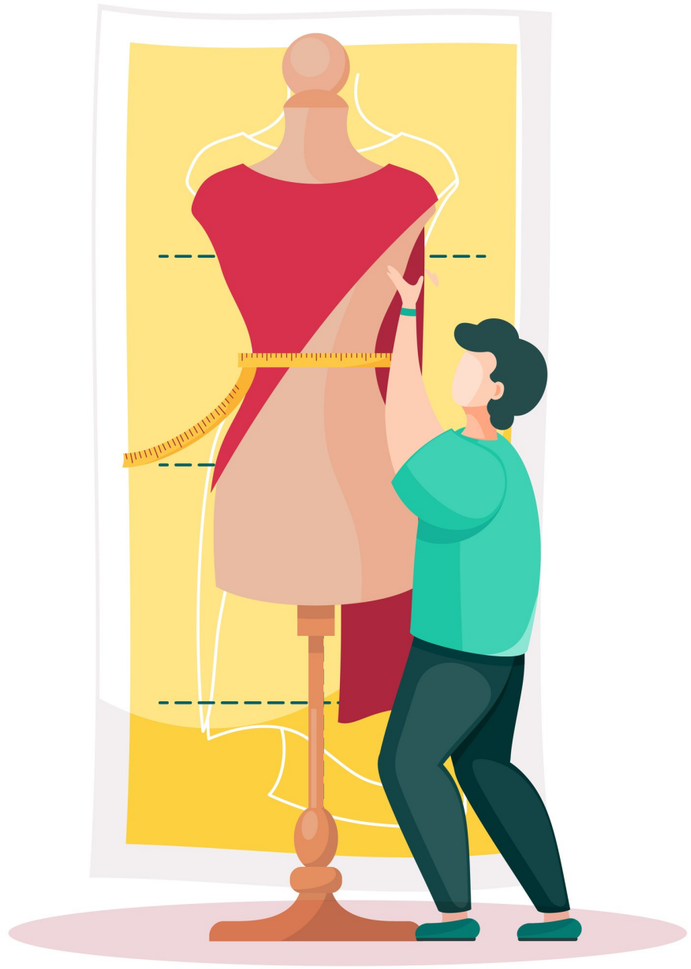
- **Demand:**
  - Gowns: At most 20 at $\$1000$
  - Tuxedos: At most 12 at $\$600$
- **Gown production:**
  - Fabric $\$110$
  - Mr. S 6 hours at $\$40/h$
  - Ms. T 3 hours at $\$35/h$
- **Tuxedo production:**
  - Fabric $\$75$
  - Mr. S 4 hours at $\$40/h$
  - Ms. T 1 hour at $\$35/h$



Gowns or tuxedos

- **Constraints:**
  - Mr. S at most 40 hours
  - Ms. T at most 20 hours

- Find the optimum number of gowns and tuxedos to maximize profit



Objective and constraints

- g: number of gowns in one week
- t: number of tuxedos in one week
- C: fabric cost + Mr. S wages + Mr. T opportunity cost
- Opportunity cost: cost of choosing to sew versus other responsibilities

$$C = 110g + 240g + 105g + 75t + 160t + 35t$$

$$C = 455g + 270t$$

Cost	Fabric	Mr. S	Ms. T
Gown	$\$110$	$\$40/h \times 6h = \$240$	$\$35/h \times 3h = \$105$
Tuxedo	$\$75$	$\$40/h \times 4h = \$160$	$\$35/h \times 1h = \$35$

Objective and constraints

- **Revenue:** $R = 1000g + 600t$
- **Cost:** $C = 455g + 270t$
- **Profit:** $\Pi = R - C = (1000g + 600t) - (455g + 270t) = 545g + 330t$

- **Constraints:**
  - Demand: $g \leq 20, t \leq 12$
  - Supply: $6g + 4t \leq 40, 3g + t \leq 20$

MILP in SciPy

```
from scipy.optimize import milp, Bounds, LinearConstraint


result = milp([-545, -330],
              integrality=[1, 1],
              bounds=Bounds([0, 0], [20, 12]),
              constraints=LinearConstraint([[6, 4], [3, 1]], ub=[40, 20]))
```

MILP in SciPy

```
print(result.message)
print(f'The optimal number of gowns produced is: {result.x[0]:.2f}')
print(f'The optimal number of tuxedos produced is: {result.x[1]:.2f}')
```

```
Optimization terminated successfully. (HiGHS Status 7: Optimal)
The optimal number of gowns produced is: 6.00
The optimal number of tuxedos produced is: 1.00
```

Integrality

```
result = milp([-545, -330],
              bounds=Bounds([0, 0], [20, 12]),
              constraints=LinearConstraint([[6, 4], [3, 1]], ub=[40, 20]))
...
```

```
The optimal number of gowns produced is: 6.67
The optimal number of tuxedos produced is: 0.00
```

Consequences of omitting integrality

- Proposed solution 6.67 gowns and 0.00 tuxedos $\rightarrow$
  - **Round** to 7 gowns and 0 tuxedos
    - Mr. S: $6g + 4t = 6 \times 7 + 4 \times 0 = 42$
    - $42 > 40$
  - **Truncate** to 6 gowns and 0 tuxedos
    - Mr. S: $6g + 4t = 6 \times 6 + 4 \times 0 = 36$
    - Ms. T: $3g + 1t = 3 \times 6 + 1 \times 0 = 18$
    - $\Pi = 545g + 330t = 545 \times 6 + 330 \times 0 = 3270$
    - Missed out $330 (nearly 10%) in profit!

Let's Practice!

INTRODUCTION TO OPTIMIZATION IN PYTHON